Daily Quant Digest: 2026-10-20

Today's topics

Track	Topic	Difficulty
Probability & Statistics	Restart, day 2: linearity of expectation and the indicator trick ("count one tally mark at a time")	Medium
Derivatives	Gamma scalping: buying low and selling high on autopilot, on a USDJPY straddle	Medium
Algorithms & DS	Graphs and topological sort: the Alien Dictionary	Medium-Hard
Music	Led Zeppelin, "Kashmir" (1975)	

1. Probability & Statistics: Linearity of Expectation and Indicators

1.1 The one idea: count one tally mark at a time

Imagine an auditor who must report "how many trades were booked wrongly today". The mistakes are tangled together: one bad system feed can break many trades at once. The full distribution of the error count would be a nightmare.

But the auditor does not need the full distribution to know the **average**. She can walk down the list, and for each trade ask one yes/no question: "is this one wrong?" Each trade gets a tally mark (1) or nothing (0). The total count is just the sum of the tally marks. So the average count is the sum of the average tally marks:

$$E[\text{count}] = \sum_{\text{trades}} P(\text{this trade is wrong}).$$

That is the whole trick. Each tally mark is called an **indicator**: $I = 1$ if the event happens, 0 if not. Its average is just its probability, $E[I] = P(\text{event})$.

The rule behind it is **linearity of expectation**:

$$E[X_1 + X_2 + \cdots + X_n] = E[X_1] + E[X_2] + \cdots + E[X_n].$$

The surprising part: **this holds even when the X_i depend on each other**. The tangled system feed does not matter. Averages add up no matter what.

Why is it true, in plain words? Picture a big spreadsheet. Each row is one possible world, weighted by its probability. Each column is one tally mark. The expected total is "add up every cell, weighted by row". You can add the cells row by row (total per world, then average) or column by column (average per column, then add). Same cells, same sum. Dependence only changes *which* cells are 1 in the same row; it never changes the grand total.

1.2 A mini example: counting the cases

Three traders throw their name badges in a box and each pulls one out at random. How many, on average, get their own badge back?

The hard way: list all $3! = 6$ ways the badges can be handed out, and count the matches in each.

Who gets which badge (A, B, C)	Own badges
A, B, C	3
A, C, B	1
B, A, C	1
B, C, A	0
C, A, B	0
C, B, A	1

Total over the 6 worlds: $3 + 1 + 1 + 0 + 0 + 1 = 6$. Average: $6/6 = 1$.

The easy way: put a tally mark on each trader. Trader A gets her own badge with probability $1/3$. Same for B and C. So the average is $1/3 + 1/3 + 1/3 = 1$.

Notice the matches are very dependent: if two traders get their own badge, the third must too. You never get exactly 2. Linearity does not care.

Now scale it up: **1,000 traders**. The full table has $1000!$ rows, a number with over 2,500 digits. The tally method still takes one line: $1{,}000 \times \frac{1}{1{,}000} = 1$. On average exactly one person gets their own badge, whether there are 3 people or a million.

1.3 The recipe

1. Write the thing you want to count as a **sum of yes/no questions**, one per small piece (a card, a position, a pair, a trader).
2. For **one** piece, find the probability the answer is "yes". Use symmetry: usually every piece looks the same.
3. Multiply by the number of pieces.

The skill is all in step 1: choosing what to put a tally mark on. Sometimes the obvious choice is hard, and a clever one is easy (you will see this in the interview question).

1.4 Common mistakes

- **Thinking dependence breaks it.** It does not, for averages. Dependence matters for the **variance** and for probabilities like "at least one", but never for the mean of a sum.
- **Mixing up "average count" and "probability of at least one".** The average number of own-badge matches is 1, but the chance that *nobody* gets their own badge is about $1/e \approx 37\%$. An average of 1 does not mean "it surely happens once".

1.5 Interview question

> Shuffle a standard 52-card deck well and deal the cards face up one by one.
> (a) On average, how many cards are dealt before the first ace appears?
> (b) On average, how many times do two neighbouring cards have the same rank (for example a 7 followed by a 7)?
> (c) On average, how many different ranks do you see among the first 10 cards?

1.6 Solution

(a) Before the first ace. The obvious approach, "the first ace is at position k with probability …", gets messy. Instead, put a tally mark on each of the **48 non-aces**: "is this card dealt before all four aces?"

Look at one non-ace, say the 9 of clubs, together with the four aces. Only the order of these **5 cards** matters for the question, and by symmetry each of the 5 is equally likely to come first. So the 9 of clubs beats all four aces with probability $1/5$.

$$E[\text{cards before first ace}] = 48 \times \frac{1}{5} = 9.6.$$

Picture it: the 4 aces cut the deck into 5 gaps (before the first ace, between aces, after the last). The 48 other cards spread over these 5 gaps, and by symmetry each gap gets the same share on average: $48/5 = 9.6$.

(b) Neighbouring pairs with the same rank. There are 51 neighbouring positions: (1,2), (2,3), …, (51,52). Put a tally mark on each. For one position, whatever the first card is, the second card is one of the other 51 cards, and 3 of them share its rank. So the probability is $3/51$.

$$E[\text{same-rank neighbours}] = 51 \times \frac{3}{51} = 3.$$

The 51s cancel. A nice answer: on average, 3 such pairs in every shuffle.

(c) Different ranks in the first 10 cards. Do not tally the cards. Tally the **13 ranks**: "does this rank show up in the first 10 cards?"

It is easier to find the chance that a rank is *missing*: all 10 cards come from the 48 cards of other ranks.

$$P(\text{kings missing}) = \frac{\binom{48}{10}}{\binom{52}{10}} = \frac{42 \cdot 41 \cdot 40 \cdot 39}{52 \cdot 51 \cdot 50 \cdot 49} \approx 0.4134.$$

(The second form comes from cancelling: the 4 kings must all land in the 42 positions after the first 10.)

$$E[\text{different ranks}] = 13 \times (1 - 0.4134) \approx 7.63.$$

Interview trick. In each part, ask "what is the small piece I can put a yes/no question on?" In (a) it was the *other* cards, in (b) the *positions*, in (c) the *ranks*. Choosing the piece is the whole problem.

Sanity checks.

- a. If there were only 1 ace, the 51 other cards split into 2 gaps: $51/2 = 25.5$ before it, which is clearly right (the ace sits in the middle on average). With 4 aces the answer must be much smaller. 9.6 fits.
- c. The answer must be below 10 (only 10 cards) and below 13. With 10 cards drawn from 13 ranks, some repeats are very likely, so about 7.6 makes sense. A quick 200,000-shuffle simulation gives 9.59, 3.00 and 7.63.

---

2. Derivatives: Gamma Scalping, Buying Low and Selling High on Autopilot

2.1 The setup: who trades this and why

Yesterday we saw the trade-off: an option holder **pays theta** (rent) every day, and **gamma pays back** when spot moves a lot. Today we see *how* gamma actually pays you. The answer is a simple routine called **gamma scalping**.

The scenario. It is the week before a Bank of Japan meeting. Our FX options desk thinks USDJPY will be jumpy, more than the options market is pricing. The desk does not know the direction. So it **buys a 1-month at-the-money USDJPY straddle**: a USD call and a USD put, both with strike 150.00, on USD 10 million each.

On the other side is, say, a fund that sells options every month to collect premium. It is happy to be the insurance company, because it thinks the market is calm.

Numbers (we ignore interest rates to keep it clean):

Item	Value
Spot	150.00
Implied vol	10%
Premium paid	about JPY 34.3 million (about USD 229,000)
Theta (rent)	about JPY 577,000 per day (about USD 3,800)
Gamma	delta changes by about USD 1.86M for each 1 yen move
Starting delta	about zero (a straddle is “half call, half put”)

2.2 The routine, with an analogy

Analogy: a market stall with a rule. You run a fruit stall with one rule: *always keep the same number of apples on the shelf*. When apple prices jump, customers rush in, you sell apples at the high price, and restock later. When prices drop, you buy apples cheaply to refill. You never predicted anything. The rule made you sell high and buy low automatically.

Gamma scalping is that rule for an option holder. The rule is: **always keep the total position delta-neutral**.

- Spot goes **up**. Because you are long gamma, your delta grows: the straddle now behaves like owning USD. To get back to neutral you **sell USD**, at the *higher* price.
- Spot goes **down**. Your delta goes negative: the straddle behaves like being short USD. To get back to neutral you **buy USD**, at the *lower* price.

Every round trip (sell high, buy back low) locks in a small profit. These profits are the “gamma payback”. The option itself is the machine that tells you how much to trade.

2.3 How big must the moves be?

The rent is paid every day. The scalping profit on a day is about

$$\text{gamma gain} \approx \tfrac{1}{2} \times \Gamma \times (\text{move})^2.$$

The breakeven move is where gain equals rent. With 10% vol:

$$\text{breakeven daily move} \approx S \times \frac{\sigma}{\sqrt{365}} = 150 \times \frac{0.10}{19.1} \approx 0.79 \text{ yen}.$$

This gives a lovely rule of thumb:

$$\text{day's P\&L} \approx \text{theta} \times \left[\left(\frac{\text{move}}{\text{breakeven}} \right)^2 - 1 \right].$$

A move of exactly 0.79 yen: break even. A move twice as big: you earn **3 days** of rent ($2^2 - 1$). No move: you lose 1 day of rent.

2.4 The worked trade: five jumpy days

USDJPY bounces around and, after five days, **ends exactly where it started**. The desk re-hedges once a day at the close.

Day	Spot	Move	New delta (USD)	Hedge trade	Day's P&L (option + hedge)
0	150.00		+0.11M	sell 0.11M	
1	151.20	+1.20	+2.33M	**sell 2.22M at 151.20**	+JPY 765k (+USD 5,060)
2	150.10	−1.10	+0.30M	**buy 2.03M at 150.10**	+JPY 522k (+USD 3,480)
3	151.50	+1.40	+2.96M	**sell 2.65M at 151.50**	+JPY 1,286k (+USD 8,490)
4	150.30	−1.20	+0.70M	**buy 2.25M at 150.30**	+JPY 733k (+USD 4,880)
5	150.00	−0.30	+0.10M	buy 0.60M at 150.00	−JPY 531k (−USD 3,540)
Total					**+JPY 2.78M (about +USD 18,500)**

Look at the hedge trades: sell at 151.20, buy at 150.10, sell at 151.50, buy at 150.30. Sell high, buy low, four times in a row.

Check day 1 with the rule of thumb. The move was 1.20 yen, and $1.20/0.79 = 1.53$. So P&L $\approx 577\text{k} \times (1.53^2 - 1) = 577\text{k} \times 1.34 \approx \text{JPY } 771\text{k}$. The exact number is 765k. Close.

The punchline. Compare with a trader who bought the same straddle and did **not** hedge. Spot ended at 150.00, the same as the start, so the straddle just lost five days of time value: about −**USD 19,900**. Same option, same market, opposite result. The difference is entirely the scalping.

Why did the hedger win? The moves averaged about 1.1 yen a day. Turned into an annual number, the market actually moved with about **14% realized volatility**, while the desk paid **10% implied**. Gamma scalping is the machine that turns "realized higher than implied" into cash.

2.5 The other side, and what goes wrong

The seller is short gamma. The same rule forces them to *buy high and sell low*. On calm days they collect rent. On wild days the hedging bleeds them. In February 2018 ("Volmageddon"), the VIX index more than doubled in a single day (5 February). Exchange-traded products that were short volatility, such as the XIV note, lost more than 90% of their value overnight and XIV was shut down. Short gamma is like selling typhoon insurance: steady income, then one very bad day.

Practical problems for the buyer:

- **Trending markets are fine; quiet markets are not.** If USDJPY rises 1 yen every day in a straight line, you still sell into each rally and still earn $\frac{1}{2}\Gamma(\text{move})^2$. What kills the long-gamma trader is a market that goes to sleep.
- **Where the moves happen matters.** Gamma is largest near the strike. Big moves while spot is far from 150 earn much less. Two paths with the same realized vol can give different P&L.
- **Hedging frequency.** Hedging more often does not raise the *average* profit much. It makes the result less random. But every hedge pays the bid-offer spread, so desks hedge on fixed spot moves (for example every 0.50 yen) or a few times a day, not every second.

2.6 Likely interview questions

1. **"You are long a delta-hedged straddle. Spot ends the week where it started. Did you make or lose money?"** It depends on the path, not the end point. If spot moved more than the breakeven each day (realized vol above implied), the scalps beat theta and you made money, as in our example.
2. **"If you hedge twice as often, do you make twice as much?"** No. The expected scalping profit depends on realized variance, which you do not change by watching more often. More frequent hedging mainly reduces the noise of the P&L, while adding transaction costs.
3. **"Rule of thumb: implied vol 16%, how big is the breakeven daily move?"** About 1% a day, because $\sqrt{256} = 16$ (traders often use about 256 business days a year). For FX quoted on 365 days, $16\%/\sqrt{365} \approx 0.84\%$.

---

3. Algorithms & Data Structures: The Alien Dictionary (Topological Sort)

3.1 Problem

An alien language uses lowercase English letters, but in an unknown order. You are given a list of words that is **already sorted** in the alien dictionary order. Return a string of all the letters that appear, in an order consistent with the list. If several orders work, return the **alphabetically smallest** one (in our English sense). If no order works, return the empty string.

Constraints: $1 \le n \le 100$ words, each of length 1 to 100, lowercase letters only.

Example 1:

```
Input:  ["wrt", "wrf", "er", "ett", "rftt"]
Output: "wertf"
```

Example 2:

```
Input:  ["z", "x", "z"]
Output: ""        (z before x and x before z: a cycle)
```

Example 3:

```
Input:  ["abc", "ab"]
Output: ""        (a longer word cannot come before its own prefix)
```

3.2 From first principles

What does a sorted list tell us? Think of how you compare two words in an English dictionary: walk along both until the **first position where they differ**. That one pair of letters decides the order. Everything after it tells you nothing.

So from `"wrt"` before `"wrf"` we learn exactly one fact: `t` comes before `f`. From `"er"` before `"ett"`: `r` before `t`. Each fact is an arrow between two letters.

Only neighbours matter. If word 1 $\leq$ word 2 and word 2 $\leq$ word 3, then word 1 $\leq$ word 3 follows automatically. So comparing each word with the next one is enough: $n - 1$ comparisons, not n^2.

One trap: prefixes. If `"abc"` comes before `"ab"`, the first difference never appears; the shorter word ran out first. In any dictionary a prefix comes first, so this list is impossible: return `""`.

Brute force. With k distinct letters, try every one of the $k!$ orders and check whether the list is sorted under it. For $k = 26$ that is about 4×10^{26} orders. Hopeless, but fine as a test reference for $k \leq 5$.

The insight: this is a graph problem. Letters are nodes. Each fact "u before v" is an arrow $u \to v$. We want to line up all nodes so every arrow points forwards. That is called a **topological order**, and it exists exactly when the graph has **no cycle**.

Analogy: getting dressed. Socks before shoes, shirt before jacket, trousers before shoes. How do you find an order? Pick anything that has no remaining "must come before it" item, put it on, cross it off the lists, repeat. If at some point every remaining item is still waiting on something, you are stuck in a loop: no valid order.

That is **Kahn's algorithm**:

1. For each letter count its **in-degree**: how many arrows point into it (how many letters must come before it).
2. Put every letter with in-degree 0 into a "ready" pile.
3. Repeatedly take a letter from the ready pile, append it to the answer, and remove its outgoing arrows. Any letter whose in-degree drops to 0 joins the ready pile.

4. If the answer has all letters, done. If some letters were never ready, they sit on a cycle: return `""`.

To get the **alphabetically smallest** answer, make the ready pile a **min-heap**: always take the smallest available letter. (Greedy is safe here: taking the smallest available letter never blocks anything, since all letters in the pile are already free to go.)

Trace on example 1. Facts: `t→f`, `w→e`, `r→t`, `e→r`. In-degrees: w 0, e 1, r 1, t 1, f 1. Ready: {w}. Take w, so e becomes ready. Take e, so r is ready. Then r, t, f. Answer `"wertf"`.

Duplicate arrows. The same fact can appear many times (`a→b` from several word pairs). If we counted it twice in the in-degree but removed it once, `b` would never become ready. We store arrows in a 26×26 yes/no table and count each one only once.

Complexity. Building the graph reads every character at most once: $O(C)$, where C is the total number of characters. Kahn's algorithm touches 26 letters and at most 26^2 arrows, with heap operations of $O(\log 26)$. Total $O(C)$ time for a fixed alphabet, and $O(26^2)$ space. For a general alphabet of size k with E arrows: $O(C + E + k \log k)$.

3.3 C++ solution

```cpp
// Daily Quant Digest 2026-10-20
// Algorithms & DS: Graphs and topological sort, "Alien Dictionary"
// Compile: g++ -std=c++17 -O2 -Wall 2026-10-20.cpp -o alien && ./alien

#include <algorithm>
#include <array>
#include <cassert>
#include <functional>
#include <iostream>
#include <queue>
#include <random>
#include <string>
#include <vector>

using std::string;
using std::vector;

// ---------------------------------------------------------------------------
// Solution: build a graph of "letter u comes before letter v" from adjacent
// words, then run Kahn's algorithm (BFS on in-degrees). A min-heap picks the
// smallest available letter each time, so the answer is the alphabetically
// smallest valid order (deterministic, easy to test).
// Returns "" if no valid order exists.
// Time O(C + 26 log 26) where C = total characters; space O(26^2).
// ---------------------------------------------------------------------------
string alienOrder(const vector<string>& words) {
    std::array<bool, 26> present{};                // letters that appear at all
    std::array<std::array<bool, 26>, 26> edge{};   // edge[u][v]: u before v
    std::array<int, 26> indegree{};

    for (const string& w : words)
        for (char ch : w) present[ch - 'a'] = true;

    // Only ADJACENT pairs are needed: if w1 <= w2 <= w3 then w1 <= w3 follows.
    for (std::size_t i = 0; i + 1 < words.size(); ++i) {
        const string& a = words[i];
        const string& b = words[i + 1];
```

```
        // std::mismatch walks both strings and stops at the first difference.
        auto [ia, ib] = std::mismatch(a.begin(), a.end(), b.begin(), b.end());
        if (ia == a.end()) continue;               // a is a prefix of b (or equal): fine
        if (ib == b.end()) return "";              // b is a proper prefix of a: impossible
        int u = *ia - 'a', v = *ib - 'a';
        if (!edge[u][v]) {                         // avoid counting a duplicate edge twice
            edge[u][v] = true;
            ++indegree[v];
        }
    }

    // Min-heap of letters with no remaining prerequisites.
    std::priority_queue<int, vector<int>, std::greater<int>> ready;
    int total = 0;
    for (int c = 0; c < 26; ++c)
        if (present[c]) {
            ++total;
            if (indegree[c] == 0) ready.push(c);
        }

    string order;
    while (!ready.empty()) {
        int u = ready.top();
        ready.pop();
        order.push_back(static_cast<char>('a' + u));
        for (int v = 0; v < 26; ++v)
            if (edge[u][v] && --indegree[v] == 0) ready.push(v);
    }
    // If a cycle exists, its letters never reach in-degree 0.
    return static_cast<int>(order.size()) == total ? order : "";
}

// ---------------------------------------------------------------------------
// Brute force reference: try every permutation of the present letters in
// alphabetical order; return the first one under which the list is sorted.
// ---------------------------------------------------------------------------
bool sortedUnder(const vector<string>& words, const string& order) {
    std::array<int, 26> rank{};
    for (std::size_t k = 0; k < order.size(); ++k) rank[order[k] - 'a'] = static_cast<int>(k);
    auto less = [&](const string& x, const string& y) {
        return std::lexicographical_compare(
            x.begin(), x.end(), y.begin(), y.end(),
            [&](char p, char q) { return rank[p - 'a'] < rank[q - 'a']; });
    };
    for (std::size_t i = 0; i + 1 < words.size(); ++i)
        if (less(words[i + 1], words[i])) return false;
    return true;
}

string bruteForce(const vector<string>& words) {
    string letters;
    for (const string& w : words) letters += w;
    std::sort(letters.begin(), letters.end());
    letters.erase(std::unique(letters.begin(), letters.end()), letters.end());
    do {
        if (sortedUnder(words, letters)) return letters;
    } while (std::next_permutation(letters.begin(), letters.end()));
    return "";
}

int main() {
    struct Case { vector<string> words; string expected; };
    vector<Case> cases = {
        {{"wrt", "wrf", "er", "ett", "rftt"}, "wertf"},   // example 1
        {{"z", "x"}, "zx"},                               // example 2
        {{"z", "x", "z"}, ""},                            // example 3: cycle
```

```
        {{"abc", "ab"}, ""},                            // example 4: bad prefix
        {{"ab", "abc"}, "abc"},                         // prefix the right way round
        {{"baa", "abcd", "abca", "cab", "cad"}, "bdac"}, // longer chain
    };
    for (const auto& c : cases) {
        string got = alienOrder(c.words);
        std::cout << "\"" << got << "\"\n";
        assert(got == c.expected);
        assert(bruteForce(c.words) == c.expected);
    }

    // Randomised cross-check: sometimes sort by a hidden alphabet (always
    // valid), sometimes leave the list in random order (often invalid).
    std::mt19937 rng(20261020);
    for (int trial = 0; trial < 20000; ++trial) {
        int L = 1 + rng() % 5;                      // alphabet a..(a+L-1)
        int n = 1 + rng() % 6;
        vector<string> words(n);
        for (auto& w : words) {
            int len = 1 + rng() % 4;
            for (int k = 0; k < len; ++k) w.push_back(static_cast<char>('a' + rng() % L));
        }
        if (rng() % 2) {
            string hidden(L, ' ');
            for (int k = 0; k < L; ++k) hidden[k] = static_cast<char>('a' + k);
            std::shuffle(hidden.begin(), hidden.end(), rng);
            std::array<int, 26> rank{};
            for (int k = 0; k < L; ++k) rank[hidden[k] - 'a'] = k;
            std::sort(words.begin(), words.end(), [&](const string& x, const string& y) {
                return std::lexicographical_compare(
                    x.begin(), x.end(), y.begin(), y.end(),
                    [&](char p, char q) { return rank[p - 'a'] < rank[q - 'a']; });
            });
            assert(alienOrder(words) != "");        // a valid order must be found
        }
        string got = alienOrder(words);
        assert(got == bruteForce(words));
        if (!got.empty()) assert(sortedUnder(words, got));
    }
    std::cout << "All tests passed.\n";
    return 0;
}
```

Output:

```
"wertf"
"zx"
""
""
"abc"
"bdac"
All tests passed.
```

3.4 Walkthrough of the C++

- **`std::array<bool, 26> present{};`** A fixed-size array on the stack. The empty braces `{}` set every element to `false` (or 0). Without them, the values would be garbage. `std::array<std::array<bool, 26>, 26>` is a 26×26 table, also zeroed by `{}`. For a tiny alphabet this beats `unordered_map<char, unordered_set<char>>`: no hashing, no heap allocation, and the whole table fits in cache.

- **`ch - 'a'`.** Characters are small integers, and `'a'` to `'z'` are consecutive, so `ch - 'a'` maps a letter to 0..25. `static_cast<char>('a' + u)` goes back the other way. `static_cast` is the explicit, searchable C++ way of converting types (instead of the C-style `(char)x`).
- **`const string& a = words[i];`** A reference is a second name for an existing object. `const` promises not to change it. No copy of the string is made. Writing `string a = words[i];` would copy every word.
- **`std::mismatch` and structured bindings.** `std::mismatch(a.begin(), a.end(), b.begin(), b.end())` walks both ranges together and returns a `std::pair` of **iterators** pointing at the first difference (or at the end of the shorter one). `auto [ia, ib] = ...` is a C++17 *structured binding*: it unpacks the pair into two named variables. An iterator is like a bookmark into a container: `*ia` reads the character it points to, and comparing with `a.end()` asks "did we fall off the end?". Use the four-iterator version of `mismatch`; the older three-iterator version reads past the end if the second string is shorter.
- **The prefix check order matters.** First `if (ia == a.end()) continue;` (the first word ran out, so it is a prefix: fine). Only then `if (ib == b.end()) return "";`. If both ran out, the words are equal and the first test catches it.
- **`std::priority_queue<int, vector<int>, std::greater<int>>`.** By default `priority_queue` is a **max**-heap. The three template arguments are: element type, the underlying container, and the comparator. `std::greater<int>` (from `<functional>`) flips it into a **min**-heap, so `top()` is the smallest letter.
- **`--indegree[v] == 0`.** Pre-decrement first lowers the count, then the comparison checks the new value. Combined with `edge[u][v] &&`, the short-circuit `&&` makes sure we only decrement for real arrows.
- **Cycle detection for free.** If the graph has a cycle, the letters on it always keep in-degree at least 1, so they never enter the heap. Comparing `order.size()` with `total` detects this without a separate DFS. (`size()` returns an unsigned `size_t`; casting to `int` avoids a signed/unsigned comparison warning under `-Wall`.)
- **Lambdas capturing by reference.** In `sortedUnder`, `[&]` means "the lambda can use local variables (here `rank`) by reference". The inner lambda `[&](char p, char q) { return rank[p - 'a'] < rank[q - 'a']; }` is a custom "less than" for letters, and `std::lexicographical_compare` uses it to compare whole words in dictionary order, exactly as the alien dictionary would.
- **`std::next_permutation`.** Rearranges a range into the next permutation in sorted order and returns `false` after the last one. Starting from the sorted string, the `do { } while (...)` loop visits all $k!$ orders, smallest first, so the first valid one is the alphabetically smallest answer. That makes it a perfect brute-force reference.
- **`std::unique` + `erase`.** `unique` moves duplicates (which must be adjacent, hence the `sort` first) to the back and returns an iterator to the new logical end; `erase` then really removes them. This "erase-unique idiom" is very common.
- **Testing.** `struct Case { ... }` with brace initialization builds a small test table. `std::mt19937` with a fixed seed gives reproducible random tests. Half of the random lists are sorted by a secret shuffled alphabet, so a valid answer must exist; the other half are random and often invalid. Every result is checked against the brute force, and every non-empty answer is checked to really sort the list.

4. Music: Led Zeppelin, “Kashmir” (1975)

Context

“Kashmir” is the closing track of side two of **Physical Graffiti**, Led Zeppelin’s double album released in February 1975. It is credited to **Jimmy Page, Robert Plant and John Bonham**. It runs for more than eight minutes, and many fans (and the band themselves) consider it one of the finest things they ever recorded.

The story behind it

The song grew from a guitar idea Page had been working on, played in an unusual open tuning (**DADGAD**), which gives it that droning, eastern sound. Page and Bonham worked up the basic track together, with Bonham’s huge, slow drum pattern underneath.

The title is a little misleading. Plant wrote the words after a long drive through a desolate desert area in **southern Morocco**, not in Kashmir. The road seemed to go on forever, and the song is about that feeling of an endless journey. Plant had never been to Kashmir when he wrote it.

John Paul Jones then took the song to another level by arranging parts for real strings and brass, played by session musicians. The result sounds like a film score crossed with a heavy rock band.

Plant has often called it one of the songs he is proudest of, and the band played it at almost every concert from 1975 onwards, including their 2007 reunion show at the O2 Arena in London. Page himself later played on Puff Daddy’s 1998 song “Come with Me” (from the Godzilla soundtrack), which is built on the “Kashmir” riff.

Why people love it

It does not sound like a normal rock song. There is no fast guitar solo and no chorus to sing along with. Instead it builds a mood: slow, heavy, unstoppable, like a caravan crossing a desert. It feels huge without being fast.

What to listen for

- **The riff that climbs.** The main guitar figure keeps rising, step by step, over a note that does not change underneath. It creates tension that never fully resolves.
- **The rhythm puzzle.** The guitar riff and Bonham’s drums do not line up the way you expect: the riff seems to repeat in groups of three while the drums keep a steady four. Tap your foot with the drums, then try following the guitar. They drift apart and meet again. This is a big part of why it feels hypnotic.
- **Bonham’s drums.** Notice how little he plays, and how heavy each hit is. He holds back, and that makes the song feel enormous.

- **The orchestra entering.** Listen for the strings and brass answering the riff. They make the second half feel like the camera pulling back to show the whole landscape.
- **Plant’s voice.** He sings almost like a storyteller in the middle sections, then soars above the band near the end.

Best on good headphones, at a volume that does it justice.